

Production Installation & Configuration Guide: ELK Stack 9.x

Comprehensive Blueprint for End-to-End Enterprise Deployment on Ubuntu/Debian Platforms

This document provides an exhaustive, step-by-step technical guide to installing, configuring, and securing an Elastic (ELK) Stack server from scratch. It details all necessary commands, architectural justifications, configuration profiles, and token generation steps required to establish a stable, production-ready monitoring environment.

Architecture Overview & Default Port Mappings

Before beginning the installation, ensure that your firewall or cloud platform's security group policies explicitly allow communication on the following essential ports:

Port	Service	Traffic Direction	Purpose
22	SSH	Inbound	Remote Server Administration
5601	Kibana	Inbound	Web User Interface & Administration Console
9200	Elasticsearch	Inbound / Internal	REST API & Core Database Communications
8220	Fleet Server	Inbound / Internal	Elastic Agent Management & Policy Orchestration

Step 1: Complete Removal of Legacy Systems

To prevent dependency conflicts or configuration overlaps, completely purge all existing Elastic components, lingering binaries, and legacy metadata from the host platform.

Run as root:

```
rm -rf /opt/elasticsearch*
rm -rf /root/elasticsearch*
rm -rf /root/kibana*
rm -rf /opt/kibana*
apt remove --purge elasticsearch kibana logstash filebeat -y
apt autoremove -y
```

```
# Forcefully remove all installation and working directories linked to Elasticsearch or Kibana
sudo rm -rf /opt/elasticsearch* /root/elasticsearch* /root/kibana* /opt/kibana*

# Purge binary packages, system configurations, and data scripts for the entire stack
sudo apt remove --purge elasticsearch kibana logstash filebeat -y

# Strip out orphaned packages and unused runtime system dependencies
sudo apt autoremove -y
```

- **rm -rf** : Evaluates targeted paths and recursively deletes file trees without prompting for user confirmation.
- **--purge** : Deletes both the application binaries and their corresponding configuration files from system storage.

Step 2: System Dependency Provisioning

Update local repository indexes and fetch transport libraries required for processing secure external software packages over HTTPS.

```
apt update
apt install apt-transport-https curl wget gnupg -y
```

```
# Synchronize local package listings with the upstream provider databases
sudo apt update

# Install transport management utilities, security validation tools, and cryptographic tools
sudo apt install apt-transport-https curl wget gnupg -y
```

- **apt-transport-https** : Enables access to data repositories secured via Transport Layer Security (TLS).
- **gnupg** : Provides GNU Privacy Guard tools needed for validating cryptographic signatures on third-party software.

Step 3: Elastic Repository Sign-On & Validation

Import the vendor's public signing key to authenticate package integrity, and add the Elastic 9.x distribution track to the host's package manager

```
$ wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | gpg --dearmor -o /usr/share/keyrings/elastic-keyring.gpg

$ echo "deb [signed-by=/usr/share/keyrings/elastic-keyring.gpg] https://artifacts.elastic.co/packages/9.x/apt stable main" | tee /etc/apt/sources.list.d/elastic-9.x.list
```

```
# Securely fetch and armor the Elastic repository verification key into a trusted keyring format
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elastic-keyring.gpg

# Append the authenticated Elastic 9.x package channel definition to your apt sources list
echo "deb [signed-by=/usr/share/keyrings/elastic-keyring.gpg] https://artifacts.elastic.co/packages/9.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elastic-9.x.list
```

cks binary distributions of cryptographic keys into a distinct file stream optimized for operational system tools.

- **tee** : Accepts standard output streams and writes them both to terminal logs and targeted storage configuration assets.

Step 4: Core ELK Stack Provisioning

Update repositories to register the newly added Elastic channel, and install the base engines.

```
apt update
apt install elasticsearch kibana logstash -y
```

```
# Refresh local indexing engines to scan the newly appended Elastic 9.x distribution files
sudo apt update

# Deploy the complete core Elastic suite onto the host operating system
sudo apt install elasticsearch kibana logstash -y
```

Step 5: Secure Elasticsearch Infrastructure Configuration

1. Network & Cluster Adaptation

Open the main configuration profile to bind network routing targets and declare system transport rules.

```
nano /etc/elasticsearch/elasticsearch.yml
```

```
sudo nano /etc/elasticsearch/elasticsearch.yml
```

Modify or append the following foundational declarations within the config

```
network.host: 0.0.0.0
http.port: 9200
transport.host: 0.0.0.0
```

2. Service Activation & Lifecycle Initialization

Reload the system supervisor daemon, enable automatic launch sequencing on system boot, and execute the runtime binary execution string.

```
sudo systemctl daemon-reload
sudo systemctl restart elasticsearch
```

```
# Re-index the unit execution structures managed by the system control supervisor sudo
systemctl daemon-reload

# Configure the Elasticsearch service to auto-start upon machine boot sudo systemctl enable
elasticsearch

# Trigger the immediate system execution of the Elasticsearch application environment sudo
systemctl start elasticsearch
```

2. Administrative Credential Provisions

Elasticsearch generates an implicit encryption layer by default during initialization. Once running, trigger an automated reset routine to establish explicit, secure credentials for the root **elastic** superuser account.

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
```

Security Record Note: Record the auto-generated password securely. For this deployment documentation profile, the administrative token is simulated as: `XyZ97_mPQa-RtB_vK92s*`

3. API Layer Validation

Verify the availability and health of the database layer using an authenticated secure connection request.

```
curl -k -u elastic:'XyZ97_mPQa-RtB_vK92s*' https://localhost:9200
```

Expected Verification Payload

```
{
  "name" : "node-production-01", "cluster_name" :
  "elasticsearch", "cluster_uuid" :
  "aBc123XyZ789_LmnOpQrstU", "version" : {
    "number" : "9.4.1", "build_flavor" : "default",
    "build_type" : "deb",
    "build_hash" : "6d82b123ac456fef7890123456789abcde",
    "build_date" : "2026-05-15T09:12:44.112233445Z",
    "build_snapshot" : false, "lucene_version" : "10.4.0",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

Step 6: Secure Kibana Configuration Integration

1. Extract Cryptographic Authority Certificates

Locate the Certificate Authority `http_ca.crt` generated during the Elasticsearch setup phase.

```
sudo find / -name "http_ca.crt" 2>/dev/null
```

Create a distinct certificate store inside the Kibana configuration directory, mirror the authority root file, and realign execution permissions to guarantee access for the system's kibana group profile.

```
sudo mkdir -p /etc/kibana/certs
sudo cp /etc/elasticsearch/certs/http_ca.crt /etc/kibana/certs/
sudo chown -R kibana:kibana /etc/kibana/certs
sudo nano /etc/kibana/kibana.yml
```

“paste” `elasticsearch.ssl.certificateAuthorities: [ "/etc/kibana/certs/http_ca.crt" ]`

2. Generate Application Service Access Credentials

Generate an administrative system credential token explicitly for the internal system communication service account **kibana_system**.

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u kibana_system
```

Security Record Note: For this guide, the generated access credential token for system interconnects is simulated as: `kib*Sys_99_vXzRtwPlm7A`

3.Establish Kibana Configuration Contexts

Open the Kibana service interface profile for editing.

```
sudo nano /etc/kibana/kibana.yml
```

Modify the file with;

```
server.host: "0.0.0.0"  
elasticsearch.hosts: ["http://localhost:9200"]
```

Step 7: System Orchestration & Service Activation

Configure the underlying system runtime processes to link the full monitoring stack together. Ensure that dependencies initialize sequentially and persist across system restarts or server failures.

```
systemctl daemon-reload  
  
systemctl enable elasticsearch  
systemctl enable kibana  
systemctl enable logstash  
  
systemctl start elasticsearch  
systemctl start kibana  
systemctl start logstash
```

```
# Re-index individual unit files to recognize recent modifications made to application system
profiles
sudo systemctl daemon-reload

# Program the core components to initialize automatically during platform startup cycles
sudo systemctl enable elasticsearch
sudo systemctl enable kibana sudo systemctl enable logstash

# Execute runtime initializations across all stack endpoints
sudo systemctl restart elasticsearch
sudo systemctl restart kibana sudo systemctl restart logstash
```

Step 8: Post-Deployment Verification Protocols

Execute the diagnostic routines below to confirm proper platform execution and connection health.

1. `systemctl status elasticsearch`
2. `systemctl status kibana`
3. `systemctl status logstash`

```
sudo systemctl status elasticsearch sudo systemctl status kibana
sudo systemctl status logstash
```

Verify that each service displays an explicit green verification status indicator: `active (running)`.

4. Monitor Network Socket Listeners

```
ss -tulnp | grep 5601
```

5. Access the Graphical User Interface

Open an external browser and navigate to your server's public IP address or DNS record mapping over port 5601:

```
http://<YOUR_IP/localhost>:5601
```

Log in using the superuser credentials generated during **Step 5**:

- **Username:** `elastic`
- **Password:** `XyZ97_mPQa-RtB_vK92s*`

Step 9: Enterprise Fleet Management Configuration

Fleet acts as a centralized administrative platform within Kibana to manage, update, and coordinate distributed data-collection agents (`Elastic Agents`).

```
++
|Elastic Agent|
+-----++
| (Port 8220) v
+-----+++

                                ++
| Web Browser +---->|Fleet Server Node+---->| Elasticsearch Cluster|
+-----++-----+++
(Port 5601) (Kibana UI) (Port 9200)
```

ADDING AGENTS TO MONITER;

- 1,Log in to your Kibana UI at--> (`http://<YOUR_IP/>localhost:5601`).
- 2.Use the top search console or expand the left-hand main navigation drawer and select **Management Fleet**.
- 3.Select **Add Fleet Server** (or **Add Agent** if initializing the deployment profile).
- 4.Fallow the instructions as given in Kibana

Maintenance & Log Management Commands

Use these administrative operational commands to manage the system environment lifecycle:

Service Manipulation Routines

```
# Restart the core tracking services
sudo systemctl restart elasticsearch kibana logstash

# Halt service runtimes
sudo systemctl stop elasticsearch kibana logstash
```

Real-Time Diagnostics & Telemetry Review

```
# Stream Elasticsearch diagnostic outputs sudo journalctl -u elasticsearch -f --no-tail

# Stream Kibana system execution logs sudo journalctl -u kibana -f --no-tail
```